

CHAPTER 8

Oracle and XML
in Action



While XML provides an enabling framework for a wide array of applications, it is only an enabling technology—it is not an application in itself. Until there is an agreed-upon schema or DTD, applications cannot use XML to reliably exchange or render data. In this chapter, we will examine two application areas in which these agreements do exist and discuss the specific implementations.

XML has been tightly linked to the Internet for a number of significant reasons. Because the content of an XML document is simply text, exchanging documents is easy over existing Internet protocols, across operating systems, and through firewalls. This capability gives rise to two major application areas—delivering content to a wide range of Internet-enabled devices and interchanging e-business data.

Before looking at these application areas in detail, we need to examine two Oracle Java components, which were introduced in Chapter 3, that make use of the XML infrastructure components: the Oracle XML SQL Utility and the Oracle XSQL Servlet.

Oracle XML SQL Utility

Many applications benefit from having their data reside in databases and querying these databases when data is required. An XML-enabled database benefits from being able to have these queries return data already marked up in XML in accordance with the database schema. The XML SQL Utility is a set of Java classes that accept these application queries, passing them through JDBC to the database and returning the resulting data in an XML format corresponding to the database schema of the query. As a complementary process, the XML SQL Utility can also accept an XML document conformant to the database schema and save the data untagged in the database across this schema.

In addition to reading and writing XML data into JDBC-enabled databases, the XML SQL Utility can create the DTD that represents the queried database schema. This DTD can then be used in application development with Oracle's Class Generators, as demonstrated in Chapter 2. Also, because the XML SQL Utility requires JDBC to connect to the database, there are separate versions for JDBC 1.1 and 1.2.

Retrieving XML-Formatted Data

Because there are similar hierarchical data relationships between them, a relational database schema can be modeled in XML. Let's assume we have a database of book listings in which a **BookList** table is set up with the following columns: BookID, Title, Author, Publisher, Year, ISBN, and Description. The following is a typical query that an application would make to the database:

```
 SELECT Title, Author, Publisher, Year, ISBN FROM BookList WHERE BookID
= 1234;
```

If this query is submitted through the XML SQL Utility, the database returns the following result:

```
 <?xml version="1.0"?>
<ROWSET>
  <ROW id="1">
    <TITLE>The Difference Between God and Larry Ellison: Inside Oracle
      Corporation</TITLE>
    <AUTHOR>Mike Wilson</AUTHOR>
    <PUBLISHER>William Morrow and Co.</PUBLISHER>
    <YEAR>1997</YEAR>
    <ISBN>0688149251</ISBN>
  </ROW>
</ROWSET>
```

This output can be in the form of a string if the application simply wants to write it to a file or in the form of a DOM object, which can be directly passed to the Oracle XML parser for transformation by its XSLT processor. Providing a DOM output eliminates a parsing operation that would otherwise need to be done before applying any XSL transformations.

As shown in the following code segment, queries can be submitted by passing them to the **oracle.xml.sql.query.OracleXMLQuery** class:

```
 /** Simple example of using Oracle XMLSQL API; this class queries the
    database with "select * from Booklist" in scott/tiger schema;
    then from the results of query it generates an XML document */

import java.sql.*;
import java.math.*;
import oracle.xml.sql.query.*;
import oracle.jdbc.*;
import oracle.jdbc.driver.*;

public class read_sample
{
  //=====
  // main() - public static void
  public static void main(String args[]) throws SQLException
  {
    String tabName = "Booklist";
    String user = "scott/tiger";
    DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
```

```
//init a JDBC connection by passing in the user
Connection conn =
    DriverManager.getConnection("jdbc:oracle:oci8:"+user+"@");

// init the OracleXMLQuery by using the initialized JDBC connection
// and passing in "Booklist" as tabName
OracleXMLQuery qry = new OracleXMLQuery(conn,"select * from "
    +tabName );

// get the XML document in the string format which allows us
// to print it
String xmlString = qry.getXMLString();

// print out the result to the screen
System.out.println(" OUTPUT IS:\n"+xmlString);

// Close the JDBC connection
conn.close();
}
}
```

An alternative command-line interface is also provided in the XML SQL Utility, which is useful in generating the DTD associated with a particular database schema. Assuming everything has been properly installed, the complete listing of command-line options is available by simply executing the following command:

```
 java oraclexml
```

The following command line illustrates how to create the DTD associated with a particular schema being queried:

```
 java oraclexml getxml -user "scott/tiger" -withDTD "SELECT *
FROM BookList"
```

For the Booklist table shown previously, the XML SQL Utility would output the following DTD embedded in the XML document produced from the query:

```
 <!ELEMENT BOOKLIST (BookID, Title, Author, Publisher, Year, ISBN,
    Description)>
<!ELEMENT BookID (#PCDATA)>
<!ELEMENT Author (#PCDATA)>
<!ELEMENT Publisher (#PCDATA)>
<!ELEMENT Year (#PCDATA)>
<!ELEMENT ISBN (#PCDATA)>
<!ELEMENT Description (#PCDATA)>
```

Saving XML-Formatted Data

Once a schema is created in the database, the XML SQL Utility can begin saving data to it as long as the XML-formatted data conforms to the DTD generated from the schema. The XML SQL Utility provides the ability to map the XML documents to table rows. The storage uses a simple mapping to map the element tag names to columns with XML strings converted to the appropriate data types through default mappings. In a case in which the XML element has child elements, it is mapped to a SQL object type.

To save the XML-formatted data, the XML SQL Utility initiates an insert statement binding all the values of the elements in the VALUES clause of the insert statement. The contents of each row element are mapped to a separate set of values.

Returning to the BookList example earlier in the chapter, the generated SQL statement to store an entry formatted in XML would be the following:

```

INSERT INTO BOOKLIST (BookID, TITLE, AUTHOR, PUBLISHER, YEAR, ISBN,
DESCRIPTION) VALUES (?, ?, ?, ?, ?, ?, ?) and BIND the values,
BOOKID -> 1234
TITLE -> The Difference Between God and Larry Ellison: Inside Oracle
Corporation
AUTHOR -> Mike Wilson
PUBLISHER -> William Morrow & Co.
YEAR -> 1997
ISBN -> 0688149251
Description -> Account of Larry Ellison;
```

The following sample code shows how this can be done from a Java program:

```

/** Simple example of using Oracle XMLSQL API; this class inserts the
data from an XML document into the database*/

import oracle.xml.sql.dml.*;
import java.sql.*;
import oracle.jdbc.driver.*;
import oracle.jdbc.*;
import java.net.*;

public class save_sample
{
    //=====
    // main() - public static void
    public static void main(String args[]) throws SQLException
    {
        String tabName = "BOOKLIST";    // table into which to insert
        String fileName = "sampdoc.xml"; // file containing the xml doc
        DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
```

```

// init a JDBC connection by passing in the user and password
Connection conn =
    DriverManager.getConnection("jdbc:oracle:oci8:scott/tiger@");

// Init the OracleXMLSave class by passing in the JDBC connection
// and the "BOOKLIST" table name.
OracleXMLSave sav = new OracleXMLSave(conn, tabName);

// Point to the XML file, sampledoc.xml, to save using a URL
URL url = sav.createURL(fileName);

// Save the data populating rows in the BOOKLIST table and
// return the number of rows written
int rowCount = sav.insertXML(url);

//Print out the confirmation of the successful insertion
System.out.println(" successfully inserted "+rowCount+
    " rows into "+ tabName);

// Close the JDBC connection
conn.close();
}
}

```

As with **getXML**, there is also a command-line version for the save function called **putXML**. This is useful for bulk-loading XML data. The following command line will load an XML document containing book listings conformant to our example DTD:

```
 java oraclexml putXML -user "scott/tiger" sampdoc.xml BookList
```

Performing Updates with the XML SQL Utility

Updates differ from inserts in that they may apply to more than one row in the table. The XML element to be updated might match more than one row if the matching columns are not key columns in the table. Therefore, updates require a list of key columns that the utility uses to identify the row to update. This is illustrated in the following example of a book listing update:

```
 <ROWSET>
  <ROW num="1">
    <BOOKID>1234</BOOKID>
    <TITLE> The Difference Between God and Larry Ellison: Inside Oracle
      Corporation </TITLE>
    <AUTHOR>Mike Wilson</AUTHOR>
    <PUBLISHER>William Morrow and Co.</PUBLISHER>
    <YEAR>1997</YEAR>
    <ISBN>0688149251</ISBN>
  </ROW>
</ROWSET>
```

```
</ROW>
</ROWSET>
```

This XML update results in the following SQL statement passing in the BookID and the key columns:

```
 UPDATE BOOKLIST SET TITLE = ?, AUTHOR = ? WHERE BOOKID = ?
    and bind the values,
    BOOKID -> 1234
    TITLE -> The Difference Between God and Larry Ellison: Inside
            Oracle Corporation
    AUTHOR -> Mike Wilson;
```

Note that we need not update all of the columns in the original XML document. The following sample code shows how this can be done from a Java program:

```
 /** Simple example of using Oracle XMLSQL API; this class updates the
    database data from an XML document submitted into the database*/

import oracle.xml.sql.dml.*;
import java.sql.*;
import oracle.jdbc.driver.*;
import oracle.jdbc.*;

public class ListUpdate
{
    //=====
    // main() - public static void
    public static void main(String argv[]) throws SQLException
    {
        String tabName = "BOOKLIST";          // table into which to insert
        DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());

        // init a JDBC connection by passing in the user and password
        Connection conn =
            DriverManager.getConnection("jdbc:oracle:oci8:scott/tiger@");

        // Init the OracleXMLSave by class passing in the JDBC connection
        // and the "BOOKLIST" table name.
        OracleXMLSave sav = new OracleXMLSave(conn, tabName);

        // init the key column used for the update
        String [] keyColNames = new String[1];
        keyColNames[0] = "BOOKID";
        sav.setKeyColumnNames(keyColNames);

        // Assume that the user passes in this document as the first argument!
        sav.updateXML(argv[0]);
        sav.close();
    }
}
```


Performing Deletes with the XML SQL Utility

The XML SQL Utility also supports deletes of XML documents. It identifies the rows to be deleted by using the key columns in the same way it does for updates. If one or more key columns are not given, it will still try to match the columns in the document. The following is the XML document and the corresponding equivalent delete SQL statement generated:

```
<ROWSET>
  <ROW num="1">
    <BOOKID>1234</BOOKID>
    <TITLE> The Difference Between God and Larry Ellison: Inside Oracle
      Corporation </TITLE>
    <AUTHOR>Mike Wilson</AUTHOR>
    <PUBLISHER>William Morrow and Co.</PUBLISHER>
    <YEAR>1997</YEAR>
    <ISBN>0688149251</ISBN>
  </ROW>
</ROWSET>
```

```
DELETE FROM BOOKLIST WHERE TITLE = ? AND AUTHOR = ? AND PUBLISHER = ? AND YEAR = ?
AND ISBN = ? AND BOOKID = ?
  binding,
  BOOKID <- 1234
  TITLE <- The Difference Between God and Larry Ellison: Inside
    Oracle Corporation
  AUTHOR <- Mike Wilson
  PUBLISHER <- William Morrow & Co.
  YEAR <- 1997
  ISBN <- 0688149251;
```

The following example shows how this delete can be done from Java, using the BookID as a key column:

```
/** Simple example of using Oracle XMLSQL API; this class deletes the
  database data from an XML document submitted into the database */

import oracle.xml.sql.dml.*;
import java.sql.*;
import oracle.jdbc.driver.*;
import oracle.jdbc.*;

public class ListDelete
{
  //=====
  // main() - public static void
  public static void main(String argv[]) throws SQLException
  {
    String tabName = "BOOKLIST";          // table into which to delete data
```



```

    DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());

    // init a JDBC connection by passing in the user and password
    Connection conn =
        DriverManager.getConnection("jdbc:oracle:oci8:scott/tiger@");

    // Init the OracleXMLSave class by passing in the JDBC connection
    // and the "BOOKLIST" table name.
    OracleXMLSave sav = new OracleXMLSave(conn, tabName);

    // init the key column used for the update
    String [] keyColNames = new String[1];
    keyColNames[0] = "BookID";
    sav.setKeyColumnNames(keyColNames);

    // Assume that the user passes in this document as the first argument!
    sav.deleteXML(argv[0]);
    sav.close();
}
}

```

Installing the XML SQL Utility

Installing the XML SQL Utility on a client or server is no different than installing any other Java library. Because it is packaged in a **.jar** archive, when using **xsu111.jar** or **xsu12.jar**, you must explicitly declare its filename and path in the CLASSPATH for your session. The XML SQL Utility depends upon the Oracle XML parser for Java v2, and its jar is included in the archive and must likewise be declared in the CLASSPATH. An **env.bat** file for Windows and an **env.csh** file for Unix are provided to help set up the session environment. Finally, the proper set of JDBC classes, 1.1 or 1.2, must be available to connect to the database.

In applications in which the XML SQL Utility will be communicating with an Oracle 8i database, an additional installation option is available. Due to the inclusion of the JServer Java Virtual Machine within 8i, you can load the XML SQL Utility jar into the database, where it can be called by internal or external procedures. Since 8.1.6 shipped with a Java 1.1-compliant JServer, you would load the **xsu111.jar**; for 8.1.7, which was Java 1.2 compliant, you would use the **xsu12.jar**. In this configuration, the utility benefits from an in-memory JDBC connection that dramatically reduces the retrieval times for large XML documents.

To assist in the installation with 8i, an **oraclexmlsqlload.bat** file for Windows and an **oraclexmlsqlload.csh** file for Unix are provided. When run, these scripts load all of the classes from the **oraclexmlsql.jar** except for the OracleXML command-line class. They will also load the XML parser for Java v2 and execute the **xmllgensql.sql** script that creates the **xmllgen** PL/SQL package and runs the test script, **oraclexmltest.sql**. This package allows the XML SQL Utility to be used from SQL or from within a PL/SQL procedure.

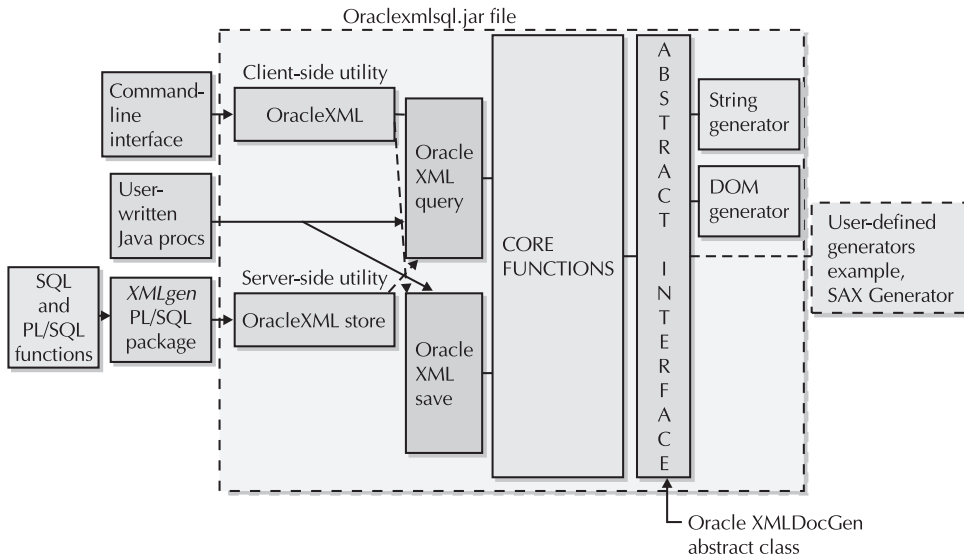


FIGURE 8-1. XML SQL Utility architecture

Extending the XML SQL Utility

While the XML SQL Utility currently supports both DOM and String outputs, it can be extended to support other forms, including SAX. Figure 8-1 shows its architecture from a function block point of view.

The core functions are wrapped with an abstract layer, **OracleXMLDocGen**, to generate the XML document. In the current implementation, this abstract class is extended by **OracleXMLDocGenDOM** to generate a DOM output, and by **OracleXMLDocGenString** to generate a String output of the XML document. Additional classes can extend the **OracleXMLDocGen** class to support other representations.

Oracle XSQL Servlet

Building upon the functionality of the XML SQL Utility and the XML parser is the XSQL Servlet introduced in Chapter 3. Written in Java, this servlet provides a high-level declarative interface to developers and Webmasters to render data across the Internet dynamically and in custom formats. Able to run with the servlet engines of most Web servers, the XSQL Servlet delivers the capability to transform a single data

source automatically in terms of the client browser and the format best suited to its capabilities and those of the platform.

The XSQL Pages

The heart of the XSQL Servlet is the XSQL page. This page is simply an XML file that contains specific elements to direct the action of the servlet. The following **booklist.xsql** file is a simple XSQL page:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="booklist.xsl"?>
<xsql:query connection="demo" xmlns:xsql="urn:oracle-xsql">
  select * from Booklist
</xsql:query>
```

Figure 8-2 shows the process flow when an Internet browser requests this page and the Web server hands over the request to the XSQL Servlet after it registers the **xsql** extension with the server. The servlet then hands the page to the XML parser to retrieve its instructions. In this case, it is asked to open a JDBC connection with the alias of **demo** and submit the query “Select * from Booklist.” It does this by passing this data to the XML SQL Utility, which performs the query as described previously and returns the result as an XML DOM object. Finally, the servlet passes the stylesheet reference along with the DOM object to the XML parser’s XSLT processor to apply the transformation to HTML for display in the client’s browser.

The essential elements of the file are the **<xsql:query>** element, which includes the database connection information within its **connection** attribute and the SQL query within its body. The connection value is an alias contained within the **<connectiondefs>** section of the **XMLConfig.xml** file:

```
<connectiondefs>
  <connection name="demo">
    <username>scott</username>
    <password>tiger</password>
    <dburl>jdbc:oracle:thin:@localhost:1521:ORCL</dburl>
    <driver>oracle.jdbc.driver.OracleDriver</driver>
  </connection>

  <connection name="xmlbook">
    <username>xmlbook</username>
    <password>xmlbook</password>
    <dburl>jdbc:oracle:thin:@localhost:1521:ORCL</dburl>
    <driver>oracle.jdbc.driver.OracleDriver</driver>
  </connection>

  <connection name="lite">
```

```

<username>system</username>
<password>manager</password>
<dburl>jdbc:Polite:Polite</dburl>
<driver>oracle.lite.poljdbc.POLJDBCdriver</driver>
</connection>
</connectiondefs>

```

This section from the default **XMLConfig.xml** file shows the declaration of the database connection string and the JDBC driver that will be used by the XML SQL Utility. As this file resides on the server in a directory not accessible to the client, this information remains secure.

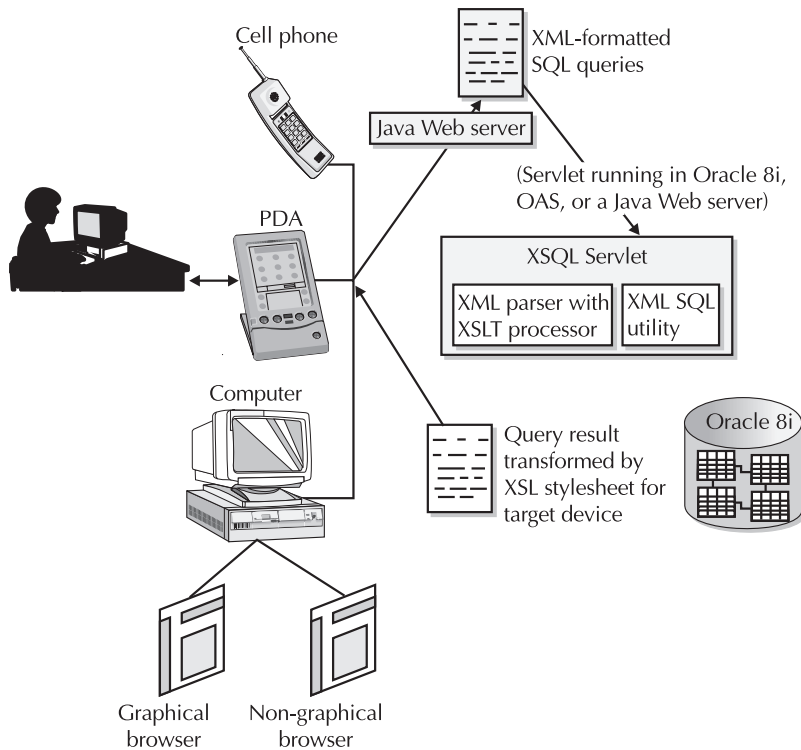


FIGURE 8-2. XSQL page process

Installing the XSQL Servlet

The XSQL Servlet is available from the OTN at technet.oracle.com/tech/xml and is designed to be quite flexible in its installation and setup. It may be used in any 1.1 or greater JVM and with any JDBC-enabled database. Specific testing has been done with JDK 1.1.8 and 1.2.2 on Windows and several Unix platforms, including Solaris, Linux, and HP-UX. The following is the list of supported and tested servlet engines:

- Allaire JRun 2.3.3
- Apache 1.3.9 with JServ 1.0 and 1.1
- Apache 1.3.9 with Tomcat 3.1 Beta1 Servlet Engine
- Apache Tomcat 3.1 Beta1 Web Server + Servlet Engine
- Caucho Resin 1.1
- NewAtlanta ServletExec 2.2 for IIS/PWS 4.0
- Oracle8i Lite Web-to-Go Server
- Oracle Application Server 4.0.8.1 (with “JSP Patch”)
- Oracle8i 8.1.7 Beta “Aurora” Servlet Engine
- Sun JavaServer Web Development Kit (JSWDK) 1.0.1 Web Server

See Chapter 9 for full instructions on installing the XSQL Servlet and the included demos.

Submitting Queries to the XSQL Servlet

The XSQL Servlet is designed to create dynamic Web pages from database queries. The XSQL pages can be linked to any Web site and can contain one or more queries whose results will replace the respective **<xsql:query>** section in the page. These results can be further customized through the use of attributes within the **<xsql:query>** tag. Table 8-1 shows the various options available.

Parameters can also be passed into the query from the HTTP request line. By prefixing an @ to the parameter name, the XSQL Servlet will search the HTTP request parameters and then the **<xsql:query>** attributes to find a match. Once a match is found, a straight lexical substitution is performed. The following is an

Attribute Name	Default	Description
rowset-element	<ROWSET>	Element name for the query results. Set equal to the empty string to suppress printing a document element.
row-element	<ROW>	Element name for each row in the query results. Set equal to the empty string to suppress printing a row element.
max-rows	Fetch all rows.	Maximum number of rows to fetch from the query. Useful for fetching the top-N rows or, in combination with skip-rows, the next-N rows from a query result.
skip-rows	Skip no rows.	Number of rows to skip over before returning the query results.
id-attribute	id	Attribute name for the id attribute for each row in the query result.
id-attribute-column	Row count value	Column name to supply the value of the id attribute for each row in the query result.
null-indicator	Omit elements with a NULL value.	If set to y or yes, causes a null-indicator attribute to be used on the element for any column whose value is NULL.
tag-case	Use the case of the column name or alias from the query.	If set to upper, UPPERCASEs the element names for columns in the query result. If set to lower, lowercases the element names for columns in the query result.

TABLE 8-1. *Attribute Options for the <xsl:query> Element*

example of an XSQL page using this function when the HTTP request line is **http://localhost/xsql/demo/booksearch.xsql?year=2001**:

```
<?xml version="1.0"?>
<xsql:query xmlns:xsql="urn:oracle-xsql" connection="demo"
  SELECT TITLE, AUTHOR, DESCRIPTION FROM BOOKLIST
  WHERE YEAR = {@year}
</xsql:query>
```

Queries that return no rows can also be handled by adding an optional **<xsql:no-rows-query>** element within the **<xsql:query>** tags. This allows the user to see a formatted page instead of the raw error. The following is an example that initially tries to retrieve the listings corresponding to the author's name; failing that, it attempts to do a fuzzy match on the submitted name:

```
 <?xml version="1.0"?>
<xsql:query xmlns:xsql="urn:oracle-xsql" connection="demo"
  SELECT TITLE, AUTHOR, DESCRIPTION FROM BOOKLIST
  WHERE AUTHOR = UPPER ('{@author}')

```

Transforming the XSQL Output with Stylesheets

The real power of the XSQL Servlet lies in its capability to dynamically transform query results by applying XSL stylesheets. The stylesheet declaration is included in the XSQL file and is applied once the XML output from the query is received. It most commonly transforms query results into HTML, as can be seen in the following example; however, the stylesheet can perform any text-based transformation. The following XSQL page and its associated stylesheet will return the results to the requesting browser as an HTML table:

```
 Searchauthor.xsql
<?xml version="1.0"?>
<xsql:stylesheet type="text/xsl" href="tortable.xsl"?>
<xsql:query xmlns:xsql="urn:oracle-xsql" connection="demo"
  SELECT Title, Author, Description FROM Booklist
  WHERE Author = UPPER ('{@author}')

```



```
</xsql:query>
```

Totable.xml

```
<html xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <head>
    <title>Book Listing</title>
  </head>
  <body>
    <table border="1" cellspacing="0">
      <tr>
        <th><b>Author</b></th>
        <th><b>Title</b></th>
        <th><b>Description</b></th>
      </tr>
      <xsl:for-each select="ROWSET/ROW">
        <tr>
          <td><xsl:value-of select="TITLE"/></td>
          <td><xsl:value-of select="AUTHOR"/></td>
          <td><xsl:value-of select="DESCRIPTION"/></td>
        </tr>
      </xsl:for-each>
    </table>
  </body>
</html>
```

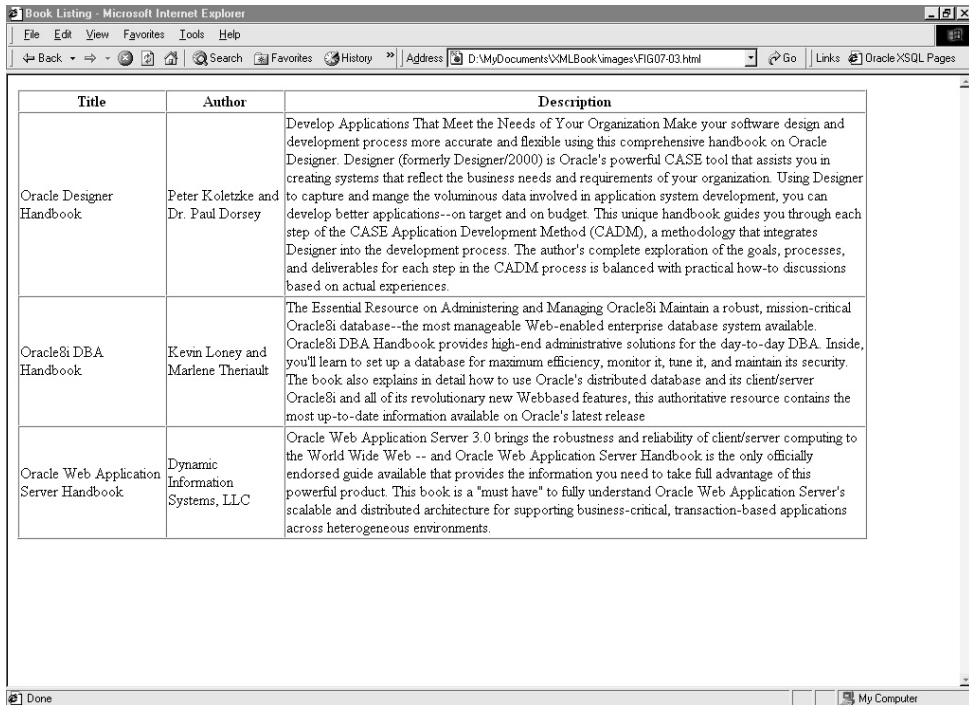
Figure 8-3 shows the query result and subsequent transformation.

Multiple stylesheet declarations are also supported. In such instances, the XSQL Servlet chooses the transformation method by matching the **user-agent** string in the HTTP header to an optional **media** attribute in the **<xml-stylesheet>** element. The match is case insensitive, and the first match in file order is the one applied. The following example shows how multiple browsers are supported.

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" media="lynx" href="booklist-lynx.xml" ?>
<?xml-stylesheet type="text/xsl" media="msie" href="booklist-ie.xml" ?>
<?xml-stylesheet type="text/xsl" href="booklist.xml"?>

<xsql:query connection="demo" xmlns:xsql="urn:oracle-xsql">
  select * from BOOKLIST
</xsql:query>
```

Note that the last stylesheet declaration has no **media** attribute. It will be applied to any HTTP requests that do not match the others and thus acts as the default



Title	Author	Description
Oracle Designer Handbook	Peter Koletzke and Dr. Paul Dorsey	Develop Applications That Meet the Needs of Your Organization Make your software design and development process more accurate and flexible using this comprehensive handbook on Oracle Designer. Designer (formerly Designer/2000) is Oracle's powerful CASE tool that assists you in creating systems that reflect the business needs and requirements of your organization. Using Designer to capture and manage the voluminous data involved in application system development, you can develop better applications--on target and on budget. This unique handbook guides you through each step of the CASE Application Development Method (CADM), a methodology that integrates Designer into the development process. The author's complete exploration of the goals, processes, and deliverables for each step in the CADM process is balanced with practical how-to discussions based on actual experiences.
Oracle8i DBA Handbook	Kevin Loney and Marlene Thenault	The Essential Resource on Administering and Managing Oracle8i Maintain a robust, mission-critical Oracle8i database--the most manageable Web-enabled enterprise database system available. Oracle8i DBA Handbook provides high-end administrative solutions for the day-to-day DBA. Inside, you'll learn to set up a database for maximum efficiency, monitor it, tune it, and maintain its security. The book also explains in detail how to use Oracle's distributed database and its client/server Oracle8i and all of its revolutionary new Webbased features, this authoritative resource contains the most up-to-date information available on Oracle's latest release
Oracle Web Application Server Handbook	Dynamic Information Systems, LLC	Oracle Web Application Server 3.0 brings the robustness and reliability of client/server computing to the World Wide Web -- and Oracle Web Application Server Handbook is the only officially endorsed guide available that provides the information you need to take full advantage of this powerful product. This book is a "must have" to fully understand Oracle Web Application Server's scalable and distributed architecture for supporting business-critical, transaction-based applications across heterogeneous environments.

FIGURE 8-3. Formatted output from the *SearchAuthor.xsql* page

stylesheet. Table 8-2 shows the allowable attributes that can be added to the `<xml-stylesheet>` element and their functions.

The final way to apply a stylesheet is to pass its URL as an HTTP parameter as follows:

 `http://localhost/yourdatapage.xsql?param1=value&xml-stylesheet=yourstyle.xml`

This technique is especially useful for prototyping and development. By replacing the stylesheet URL with **none**, you ensure that the raw XML document is sent without any stylesheet processing.

Attribute	Required	Description
type	Yes	Must be set to the value text/xsl ; otherwise, the <?xml-stylesheet?> instruction is ignored by the XSQL Page Processor.
href	Yes	Indicates the relative or absolute URL of the stylesheet to be used.
media	No	If set, this attribute's value is used to perform a case-insensitive match on the User-Agent string of the requesting browser, so the appropriate stylesheet is used, depending on the requesting software/device.
client	No	If set to yes , will download the raw XML to the client and include the <?xml-stylesheet?> processing instruction for processing in the browser. If not specified, the default is to perform the transform in the server.

TABLE 8-2. *Attribute Options for the `<xml:stylesheet>` Element*

Inserting XML Documents with the XSQL Servlet

Leveraging the full capability of the XML SQL Utility, an XSQL page can also be set up to insert XML documents into a database. An XML document can be submitted to the **OracleXMLSave** class of the XML SQL Utility by employing the Action Element, **<xsql:insert-request>**. As discussed previously, the schema must already exist in the database to save a document. While at first this may be considered a limitation, the XSQL Servlet's capability to apply a stylesheet to the XML document on submission provides the necessary functionality to filter or transform documents as needed.

Returning to the book listing example presented earlier in the chapter, an XSQL page can be set up to accept book listings not only in the prescribed format of the database but also from virtually any text-based format. For example, consider the case in which a local bookseller would like to list his books on the pages, but his book listings use a different set of tags from the database schema. By creating an XSL stylesheet and applying it on receipt of his listings, his selections could be accommodated. The following XSQL page could accept the book feed from "Joe's Books" via HTTP and transform it into the Booklist database schema by applying the **joesbooks.xsl** stylesheet and then submitting the resulting XML to the **OracleXMLSave** class for insertion:

```
<?xml version="1.0">
<xsql:insert-request xmlns:xsql="urn:oracle-xsql"
    connection = "demo"
    table = "BOOKLIST"
    transform = "joesbooks.xsl"/>
```

However, one more item must be set up to allow this example to function properly. The database generates the BookID column; therefore, this column's entry must be created for each new insertion. This can be done by setting up a trigger on the Booklist table that generates this ID whenever an insertion is made. The following SQL script will create a new BookID when each new listing is added, assuming you have already created a sequence named **bookid_seq**:

```
CREATE TRIGGER booklist_autoid
BEFORE INSERT ON BOOKLIST FOR EACH ROW
BEGIN
    SELECT bookid_seq.nextval
    INTO :new.BookID
    FROM dual;
END;
```

Other Action Elements that are supported by the XSQL Servlet and their functions are listed in Table 8-3.

Action Element	Description
<xsql:query>	Execute an arbitrary SQL statement and include its result set in canonical XML format.
<xsql:dml>	Execute a SQL DML statement or PL/SQL anonymous block.
<xsql:set-stylesheet-param>	Set the value of a top-level XSLT stylesheet parameter. The parameter's value can be set by supplying the optional value attribute or by including a SQL statement as the element content.
<xsql:insert-request>	Insert the (optionally transformed) XML document that has been posted in the request into a database table or view. If HTML form has been posted, then the posted XML document is materialized from HTTP request parameters, cookies, and session variables.

TABLE 8-3. *Action Elements and Their Functions for XSQL Pages*

Action Element	Description
<xsql:include-xml>	Include arbitrary XML resources at any point in your page by relative or absolute URL.
<xsql:include-request-params>	Include key information such as HTTP parameters, session variable values, and cookies in your XSQL page for addressing them in your stylesheet.
<xsql:include-xsql>	Include the results of one XSQL page at any point inside another.
<xsql:include-owa>	Include the results of executing a stored procedure that makes use of the <i>Oracle Web Agent (OWA)</i> packages inside the database to generate XML.
<xsql:action>	Invoke a user-defined action handler, which is implemented in Java, for executing custom logic and including custom XML information into your XSQL page.
<xsql:ref-cursor-function>	Include the canonical XML representation of the result set of a cursor returned by a PL/SQL stored function.
<xsql:set-page-param>	Set a page-level (local) parameter that can be referred to in subsequent SQL statements in the page. The value can be set using a static value, the value of another parameter, or the results of a SQL statement.
<xsql:include-param>	Include a parameter and its value as an element in your XSQL page.
<xsql:set-session-param>	Set an HTTP session-level parameter. The value can be set using a static value, the value of another parameter, or the results of a SQL statement.
<xsql:set-cookie>	Set an HTTP cookie. The value can be set using a static value, the value of another parameter, or the results of a SQL statement.
<xsql:insert-param>	Insert the value of a single parameter containing XML. Can optionally supply a transform to get it in canonical format.

TABLE 8-3. *Action Elements and Their Functions for XSQL Pages (continued)*

Updating Data with the XSQL Servlet

Many applications require that data or documents be updated instead of wholly replaced. In a similar manner to the way the BookID was automatically generated, you can use a form of trigger to provide this functionality.

Oracle 8i makes available an INSTEAD OF trigger, which allows a stored procedure in PL/SQL or Java to be called whenever an INSERT of any kind is attempted. These triggers utilize Oracle 8i's Object Views to be associated with the INSERT.

For example, if you wanted to have the Booklist table be updatable, you could initially search for the unique combination of title and author and, if one is found, perform an UPDATE instead of doing an INSERT. To set this up, you must create an Object View corresponding to the Booklist table. This can be done using the following SQL:

```
 CREATE VIEW Booklistview AS
SELECT * FROM Booklist;
```

Next, the trigger needs to be created and associated with this view. In this example, PL/SQL is being used, but the job could also be done with a Java stored procedure.

```
 CREATE OR REPLACE TRIGGER insteadOfIns_booklistview
INSTEAD OF INSERT ON booklistview FOR EACH ROW
DECLARE
    notThere BOOLEAN := TRUE;
    tmp       VARCHAR2(1);
    CURSOR chk IS SELECT 'x'
                   FROM BOOKLIST
                   WHERE TITLE = :new.title
                   AND AUTHOR = :new.author;
BEGIN
    OPEN chk;
    FETCH chk INTO tmp;
    notThere := chk%NOTFOUND;
    CLOSE chk;

    IF notThere THEN
        UPDATE INTO Booklist(TITLE,
                              AUTHOR,
                              PUBLISHER,
                              YEAR,
                              ISBN,
                              DESCRIPTION)
```

```

VALUES (:new.title,
        :new.author,
        :new.Publisher,
        :new.Year,
        :new.ISBN,
        :new.Description);
END IF;
END;
```

Finally, the XSQL file needs to be changed as follows to update the Booklistview instead of the Booklist table.

```

<?xml version="1.0">
<xsql:insert-request xmlns:xsql="urn:oracle-xsql"
    connection = "demo"
    table = "Booklistview"
    transform = "joesbooks.xsl"/>
```

As a final note, since the uniqueness is being checked in terms of the combination of the title and author, a unique index can be created to speed up the check and improve performance. The following SQL statement will create the index:

```
CREATE UNIQUE INDEX booklist_index ON booklist(Title, Author);
```

While the previous example explained how to perform an update using the trigger functionality of an object view, you can also use the XML SQL Utility update capability from the XSQL Servlet using the following very simple **.xsql** file:

```

<?xml version="1.0"?>
<xsql:dml connection="demo" xmlns:xsql="urn:oracle-xsql">
    update Booklist set status='S' where BookID = '1';
</xsql:dml>
```

This example illustrates the simplicity and power of the XSQL's XML-based interface.

An XML-Powered Web Site

Traditional Web sites have had to have their content preformatted for an anticipated screen size with little in the way of automatic client detection and customization possible. Most sites that try to accommodate different browsers are limited to a user selecting between “frames” and “non-frames” or “text” and “text+images” versions. These pages are typically stored statically and must be updated manually, introducing the possibility of errors or missing data. Any changes in formatting has to be replicated across every page.

The introduction of cascading stylesheets improved this situation by off-loading much of the HTML formatting to a separate file. While this support removes much of the formatting from the source HTML file, it still depends upon a static source file with HTML formatting. To deliver true data sources, rendered dynamically through stylesheets to browsers, has been the goal of many e-commerce companies hoping to speed the delivery of their current data. Pursuit of this strategy has resulted in enterprise-level databases such as Oracle being added to Web site backends.

The XML-Enabled Solution

In Chapter 3, we introduced the use of XML's capability to describe data in a structured way coupled with XSL's capability to perform virtually any text-based transformation on that data, delivering the solution of a true, single, unified data repository. This data repository can serve as a backend for a presentation system, be it a browser, PDA, cell phone, or pager. Embedded links, instead of calling static Web pages, can generate database queries returning up-to-the-minute content.

On the backend, databases can accept a variety of data feeds, including those formatted in XML, and store the data for immediate use. Content repositories can also be created containing reusable document fragments or modules that can be assembled by XSL stylesheets on demand to vastly simplify online document publishing. This content and presentation separation also simplifies the production process, as presentation designers can work on the stylesheets without the risk of introducing content errors.

A real-world example of such a Web site is a real-estate aggregation site where listings from a variety of *multiple listing services (MLSs)* can be assembled in one place. The scenario is that you have a Web site that accepts these MLS listings from a variety of sources, each with its own XML format. An XSL stylesheet unique to the source is applied to the XML feed, which is then transformed to a normalized format for insertion into a database. A summary of this data is presented to requesting browsers that can then search for or expand a particular listing.

The Design Requirements

To properly design this Web site, two general design requirements must be met: The database must have the capability to accept XML and do the appropriate XSL transformation for the source. It also must be able to dynamically render the listings in a variety of ways in a scalable manner with high performance.

The system must also be able to handle a variety of image formats and be able to select the appropriate type for display on the requester's system or display no images at all in the case of a cell phone or pager. The system would ideally not only respond to direct requests but also generate messages in response to receiving listings that met preset criteria. This can be especially useful to real-estate agents searching for new listings to present to clients.

The Architecture

This system is ideally suited to be designed with a database as its repository. In this case, you can use Oracle 8i to store both the data and the images, if required. Object views are set up to link the data with the images to present a complete listing to Internet browsers after XSL transformation with the appropriate stylesheet.

Figure 8-4 shows a diagram of the overall architecture of the real-estate site. XML data feeds are accepted by the XSQL Servlet, transformed into a normalized format, and saved to the database.

Clients using a browser can search the listings or display available listings in accordance with several preset criteria. The format is automatically keyed to the requesting browser's capabilities. Additionally, notifications can be sent out in the event of a change in the status of or an update to a listing.

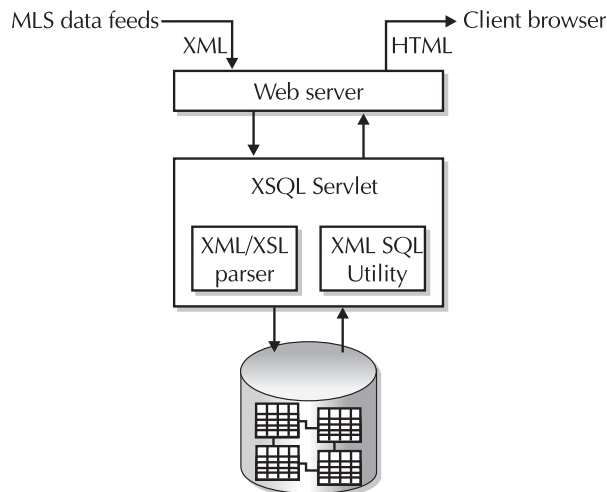


FIGURE 8-4. *Function diagram of the Real Estate site*

An Example Implementation

To implement this system, you must first design a normalized MLS schema. For the purpose of this example, the following DTD will represent the data model:

```
<img alt="book icon" data-bbox="104 213 166 231"/>
<!ELEMENT LISTING (ListingID, MLS_Number, Address, Area, City, State,
                    Zipcode, Description, AskPrice, Agent, ImageURI,
                    Category, Status)>
<!ELEMENT ListingID (#PCDATA)>
<!ELEMENT MLS_Number (#PCDATA)>
<!ELEMENT Address (#PCDATA)>
<!ELEMENT Area (#PCDATA)>
<!ELEMENT City (#PCDATA)>
<!ELEMENT State (#PCDATA)>
<!ELEMENT Zipcode (#PCDATA)>
<!ELEMENT Description (#PCDATA)>
<!ELEMENT AskPrice (#PCDATA)>
<!ELEMENT Agent (#PCDATA)>
<!ELEMENT ImageURI (#PCDATA)>
<!ELEMENT Category (SFH | CONDO | APTBLDG | DUP | THM | COM | LOT)>
<!ELEMENT Status (A | P | S | C)>
```

This DTD can serve as the template to create the stylesheets that are keyed to the specific MLS services. For example, if the NorCal service used **Neighborhood** instead of **Area**, and **Zip** instead of **Zipcode**, the following stylesheet could perform the transformation:

```
<img alt="book icon" data-bbox="104 577 166 595"/>
<?xml version="1.0"?>
<ROWSET xmlns:xsl=http://www.w3.org/1999/XSL/Transform
        xsl:version="1.0">
  <xsl:for-each select="Norcal/LISTING">
    <ROW>
      ...
      <Area><xsl:value-of select="Neighborhood"/></Area>
      ...
      <Zipcode><<xsl:value-of select="Zip"/>></Zipcode>
      ...
    </ROW>
  </xsl:for-each>
</ROWSET>
```

Once all of the stylesheets have been created, an XSQL page can be set up to accept each service and apply the appropriate transformation, as shown in the following XSQL page, which uses the `<xsql:insert-request>` Action Element described in the earlier section “Inserting XML Documents with the XSQL Servlet.”

```
<?xml version="1.0"?>
<xsql:insert-request xmlns:xsql="urn:oracle-xsql"
    connection = "demo"
    listingview = "LISTING"
    transform = "norcal.xsl"/>
```

XSQL pages can also be used on the client side to query the listings and produce various pages, such as a summary page based on **Area** or **Zipcode**, as well as refining the searches with price ranges or property type, as shown in Figure 8-5.

Real Estate Search Page
Select the criteria to perform your search

Criteria	Value	Description
Maximum Price	350,000	Enter the maximum price of listings to be included.
Category	Single Family Home	Enter the type of listings to be included.
Area	North Beach	Enter the neighborhood in which that the listings should be limited.
Zip	94000	Enter the zipcode in which the listings should be limited.
City	San Francisco	Enter the city in which the listings should be limited.
State	CA	Enter the state in which the listings should be limited.

FIGURE 8-5. Sample MLS listing search page

You can perform these searches by creating links to XSQL pages that perform the specific queries or by using the **@parameter** technique to pass the criteria to a generic XSQL page.

With the proliferation of Web-enabled cell phones, PDAs, and pagers, this site can also meet their unique and limited display requirements by utilizing the **user-agent** string in the HTTP header and the **media** attribute in the stylesheet declaration. The following XSQL example supports the Palm Pilot, the Unwired Planet cell phone, and standard browsers:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" media="Palm" href="ListingPP.xsl"?>
<?xml-stylesheet type="text/xsl" media="Mozilla" href="ListingMZ.xsl"?>
<?xml-stylesheet type="text/xsl" media="UP" href="ListingWML.xsl"?>
<?xml-stylesheet type="text/xsl" href="Listing.xsl"?>

<xsql:query connection="demo" xmlns:xsql="urn:oracle-xsql">
  select * from Listingview WHERE
    Zipcode=94065
</xsql:query>
```

Extending the Example

Using the INSTEAD OF INSERT trigger described in the earlier section “Updating Data with the XSQL Servlet,” you can make this Web site proactive by having it notify registered users of new listings, pending sales, and closings as they are inserted or updated. The stored procedure called by the trigger could query an interest registration table of e-mail addresses and send a broadcast e-mail to those registered for a particular listing, area, zip code, and so on.

A full-service site could link in financial information to alert clients or agents to loan approvals, title clearances, and other notifications during the real-estate transaction. Finally, with the upsurge in home auctions, this site could be extended to support hosting an auction service, accepting bids and notifying bidders across a wide array of communication devices.

Oracle Portal-to-Go

A product that already implements much of the functionality described in this example is Oracle’s Portal-to-Go. Primarily designed for wired and wireless service providers, Portal-to-Go permits a single content repository to serve a wide array of devices, utilizing gateways to provide the custom formatting specific to the requesting device.

Figure 8-6 displays the architecture of Portal-to-Go. XML-formatted information feeds are connected to an Oracle 8i database, storing content in a normalized manner. Oracle9i Dynamic Services, which crawls Web sites, and returning data in XML, can deliver additional feeds.

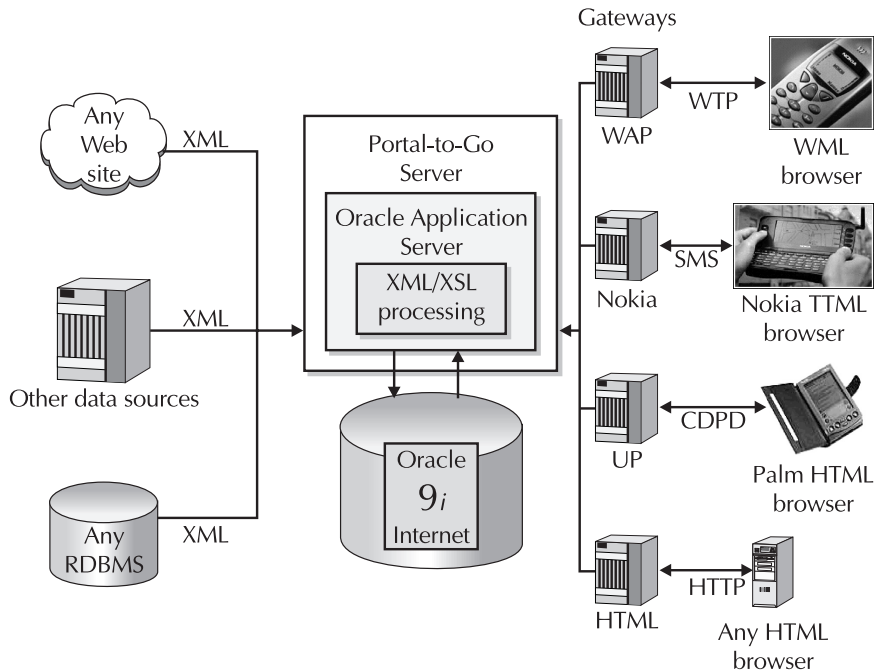


FIGURE 8-6. *Oracle Portal-to-Go architecture*

On the output side, device-specific gateways are set up to allow intelligent interaction with a variety of two-way wired and wireless devices. Users can customize their own views and content selections. Through the use of portlets, that are well defined areas of HTML/XML in a web page providing basic access to an information resource, users can even operate in disconnected mode, which is ideal for many wireless users.

XML-Powered Messaging for E-Business

The requirement to exchange data between applications and enterprises is growing daily. Historically, this data exchange required a high level of technological agreement about data types, structures, transport protocols, and so forth. Services such as CORBA delivered this capability but were limited in their capability to use the Internet, pass through firewalls, and set up ad hoc sessions. What was needed was a method for

applications to send and receive data in text form across a variety of transports and protocols and in a self-describing manner. Given this, applications would not need rigorous adherence to a technology stack to communicate. This would then allow companies to integrate applications across the Internet, meeting the demands of industry-consolidation trends and opening a whole new service area for application hosting.

The XML-Enabled Solution

The rise of XML and its related technologies of XSL, XML Schema, XML Query, and XPath make the possibility of true application integration across enterprises a reality. Whether by purchase order or update message, applications need to exchange structured data. XML-formatted data eliminates the need for metadata and its structure to be inferred through hard-coded application logic. XML Schema definitions or DTDs can be included to validate the messages. XSL stylesheets can transform these messages into specific application-compliant formats. Finally, all of this processing can be done by generic components built to the open standards. These standards give the companies that use them to develop products the confidence that they will be widely supported and used.

Because this is such a widespread application area, let's return to the Booklist example and create an online bookstore that utilizes these various XML technologies. The scenario is that we have an online used bookstore for which we would like to both receive listings of books for sale and permit clients to purchase books over the Internet. We would also like to aggregate the book listings from other used bookstores to promote to a national clientele.

The Design Requirements

To properly design this Web site and application, we need to design data models for the different objects that will be utilized. Because the database is going to serve as primary storage and is the most constrained component when it comes to modeling, we should design the database schema first. From there, the XML SQL Utility is used to generate the DTDs that map properly. Then the Class Generator converts these DTDs to the DOM classes that are used in the application front end to create the XML documents that will be inserted by the XML SQL Utility into the database.

The system must be able to accept book listings from individual sellers as well as bulk listings from other booksellers. It also needs to present book listings to prospective buyers, process sales transactions, and notify sellers. Finally, it must handle payments from buyers and to sellers.

Consideration must be given to the overall performance in the design requirements. While the retrieval of the data in XML format scales well, complex transformations and the requirement of one thread per stylesheet instance can produce a significant bottleneck under heavy loads. Minimizing the stylesheet

processing and employing middle-tier caches, such as Oracle's iCache, can mitigate much of this bottleneck.

The Architecture

This system can be implemented using Oracle 8i as the data repository and Oracle Application Server as both the Web server and the middle tier. Clients can interact with the site using standard browsers. Booksellers will be able to submit XML-formatted listings and receive sales notifications. Figure 8-7 shows a functional diagram of the system.

To insert a book listing into the store, a seller enters data into a *Java Server Page (JSP)* that calls the DOM classes to generate the XML document to be inserted. This document is then parsed and its data inserted into the Booklist schema. Additionally, the seller's account information is inserted into an Account schema. Clients are presented with the book listings by category or as the result of searches by interacting with HTML pages generated from the XSQL Servlet. Book purchases capture and store the transaction in the database and notify the seller via e-mail.

While this is by no means a complete application and architecture, the discussion and example implementation of its various parts serve as a good representation of the use of XML and as a foundation for many similar applications.

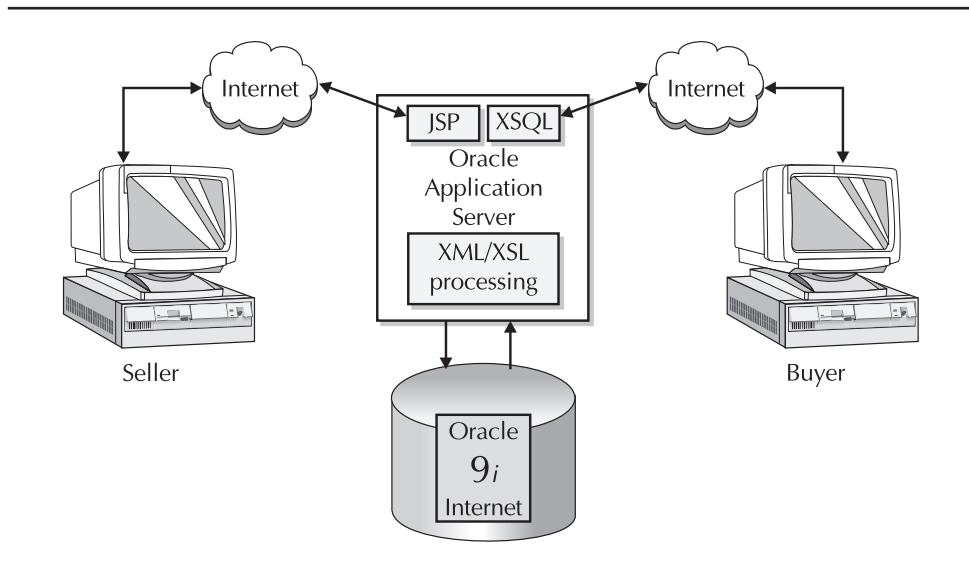


FIGURE 8-7. Functional diagram of online bookstore

An Example Implementation

By extending the Booklist schema introduced earlier in the chapter and adding a Client schema, we can discuss the specifics of the implementation. The following is the XML DTD that represents the extended book listing:

```
<!ELEMENT BOOKLIST (BookID, Title, Author, Publisher, Year, ISBN,
                    Description, Category, Cost, Status)>
<!ELEMENT BookID (#PCDATA)>
<!ELEMENT Title (#PCDATA)>
<!ELEMENT Author (#PCDATA)>
<!ELEMENT Publisher (#PCDATA)>
<!ELEMENT Year (#PCDATA)>
<!ELEMENT ISBN (#PCDATA)>
<!ELEMENT Description (#PCDATA)>
<!ELEMENT Category (FICTION | NONFICTION | REFERENCE | TECHNICAL)>
<!ELEMENT Cost (#PCDATA)>
<!ELEMENT Status (A | S)>
```

This DTD can then serve as input to the Class Generator to create the set of DOM classes that can create conformant XML documents. The following is an example command line:

```
Java samplemain booklist.dtd
```

It will create Java source files for all of the elements, which can then be compiled into Java classes using the Javac compiler. Next, a JSP is created using these classes, as illustrated in the following listing:

```
<HTML>
<HEAD>
  <TITLE>Book Submission</TITLE>
</HEAD>
<BODY bgcolor = "#ffffff">
  <%@ page import="java.io.*" %>
  <%
String Title = request.getParameter("TITLE");
String Author = request.getParameter("AUTHOR");
String Publisher = request.getParameter("PUBLISHER");
String Year = request.getParameter("YEAR");
String ISBN = request.getParameter("ISBN");
String Description = request.getParameter("DESCRIPTION");
String Category = request.getParameter("CATEGORY");
String Cost = request.getParameter("COST");
String status = new String("A"); // A for available
```



```

try {
    out.println(author + " " + TITLE + " " + CATEGORY + " " + ISBN
        + " " + PUBLISHER + " " + YEAR + " " + DESCRIPTION
        + " " + COST + " " + STATUS);

    Booklist entry = new Booklist();
    oracle.xml.parser.v2.DTD dtd = entry.getDTDNode();
    TITLE t = new TITLE(TITLE);
    AUTHOR a = new AUTHOR(AUTHOR);
    PUBLISHER p = new PUBLISHER(PUBLISHER);
    YEAR y = new YEAR(YEAR);
    ISBN i = new ISBN(isbn);
    CATEGORY c = new CATEGORY(CATEGORY);
    COST ct = new COST(COST);
    STATUS st = new STATUS(STATUS);

    entry.addNode(t);
    entry.addNode(a);
    entry.addNode(p);
    entry.addNode(y);
    entry.addNode(i);
    entry.addNode(c);
    entry.addNode(ct);
    entry.addNode(st);
    entry.print(new FileOutputStream("result.xml"));
}
catch (Exception e) {
    out.println("..." + e + "..e.");
    e.printStackTrace(new PrintWriter(out));
}
%>
</BODY>
</HTML>

```

Using JavaScript and providing a fill-in form interface, this JSP can be called from an HTML page. The following HTML page is an example of this; it produces the book submission form shown in Figure 8-8.

```

<html>
<head>
    <title>Book Submission Form</title>
</head>
<body bgcolor="#ffffff">
<script language="JavaScript">
    function checkRequiredFields() {
        var frmR = window.document.frmReport;
        if (frmR.title.value == "") {

```



```

        <td><input type="text" name="TITLE" value size="60"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>Author</td>
        <td><input type="text" name="AUTHOR" value size="60"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>Publisher</td>
        <td><input type="text" name="PUBLISHER" value size="60"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>Year</td>
        <td><input type="text" name="YEAR" value size="4"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>ISBN</td>
        <td><input type="text" name="ISBN" value size="20"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>
            Description</td>
        <td><input type="text" name="DESCRIPTION" value size="200"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>Category</td>
        <td><input type="text" name="CATEGORY" value size="12"></td>
    </tr>
    <tr>
        <td><div align="left"><p><font color="red">*</font>Cost $</td>
        <td><input type="text" name="COST" value size="6"></td>
    </tr>
</table>
<p><br>
    <input type="button" value="Submit" onClick=
        "checkRequiredFields();">
    <input type="reset" value="Reset">
</p>
</form>
</body>
</html>

```

The screenshot shows a web browser window titled "Book Submission Form - Microsoft Internet Explorer". The address bar shows the path "D:\MyDocuments\XMLBook\images\FIG07-08.html". The page content is titled "Oracle Used Books Store". Below the title is a section labeled "Seller Form" containing a form with the following fields:

*Title	Oracle and XML
*Author	John Smith
*Publisher	Oracle Press
*Year	2001
*ISBN	123
*Description	This book details how Oracle products can be used with XML and related technologies. It provides working examples with sample code.
*Category	Technical
*Cost \$	49.00

At the bottom of the form are two buttons: "Submit" and "Reset".

FIGURE 8-8. *Example book submission form*

The following is an example of the XML document produced by the JSP and passed to the XML SQL Utility for insertion:

```
<?xml version="1.0"?>
<Booklist>
  <BookID>001234</BookID>
  <Title>The Difference Between God and Larry Ellison: Inside Oracle
    Corporation</Title>
  <Author>Mike Wilson</Author>
```

```

<Publisher>William Morrow & Co.</Publisher>
<Year>1997</Year>
<ISBN>0688144251</ISBN>
<Description>Account of Larry Ellison</Description>
<Category>Computer</Category>
<Cost>30.00</Cost>
<Status>A</Status>
</Booklist>

```

To design the buyer Web page, we use an XSQL page to initially represent a query based on the categories available, as shown in Figure 8-9. Each link could call a specific page as follows, or the parameter capability of the servlet can pass the category to a generic page.

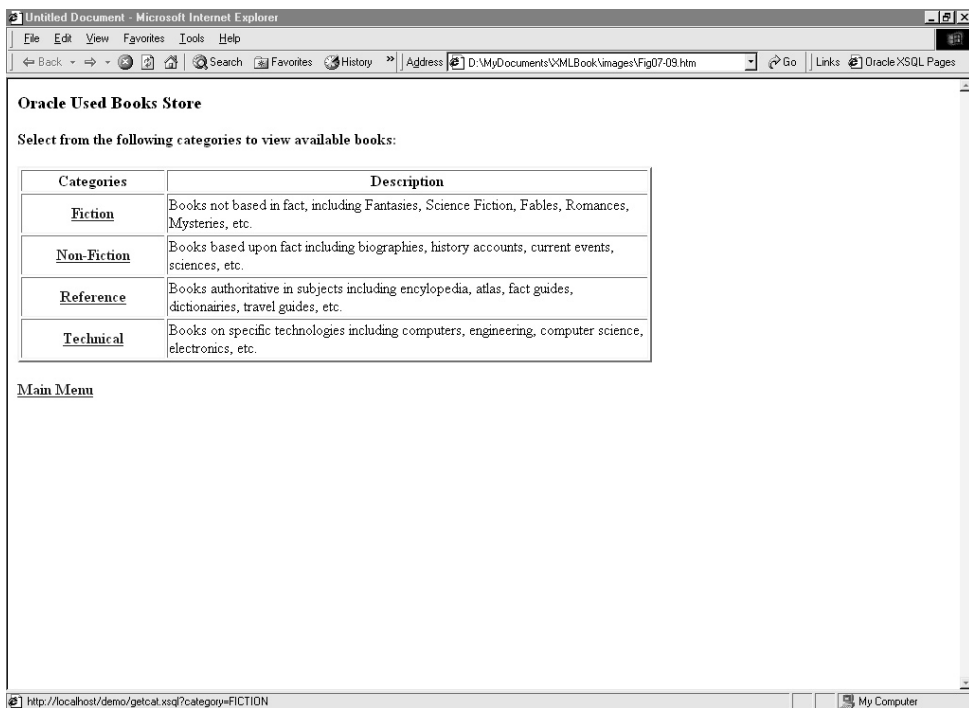


FIGURE 8-9. Bookstore category submission page

```

<?xml version="1.0"?>
<xsqlstylesheet type="text/xsl" href="categorydetail.xsl"?>
<xsql:query xmlns:xsql="urn:oracle-xsql" connection="demo"
    SELECT Title, Author, Description, Cost FROM Booklist
    WHERE Category = FICTION
</xsql:query>

```

The **categorydetail.xsl** stylesheet can then format the display of listings and add the form action to allow for the purchase. By utilizing the update capability detailed previously, the Status column can be changed from *A*, available, to *S*, sold, and the trigger procedure could be extended to send an e-mail notification of the sale by using the **utl_smtp** package.

The remaining task is to set up a form to create accounts on the system and keep track of submissions and sales. You can do this using the XML SQL Utility, Class Generator, and JSP architecture of the book submission. The implementation is left to the reader, but the following example XML file is provided to get started:

```

<Client>
  <ClientID>123456</ClientID>
  <Name>Mike Wilson</Name>
  <Address>123 Main St.</Address>
  <City>San Francisco</City>
  <State>CA</State>
  <Country>USA</Country>
  <Zipcode>94000</Zipcode>
  <Email>mwilson@anywhere.com</Email>
  <Date>03-MAY-2002</Date>
  <Time>22:00</Time>
  <Status>A</Status>
  <Account>
    <Buy>
      <BookID>103454</BookID>
      <Date>04-JUN-2003</Date>
      <Time>11:05</Time>
    </Buy>
    <Sell>
      <BookID>001234</BookID>
      <Date>03-MAY-2002</Date>
      <Time>22:00</Time>
    </Sell>
  </Account>
</Client>

```

Extending the Example

The bookstore example, while simple, can be easily extended in a number of areas. On the client side, you could add a search function that uses the “fuzzy match” capability described previously. Java or PL/SQL stored procedures could call into an authorization service to verify credit information and return a card approval number. By accepting XML book listings from other vendors, the site could act simply as an aggregator, leaving it up to the sellers to actually process the shipment.

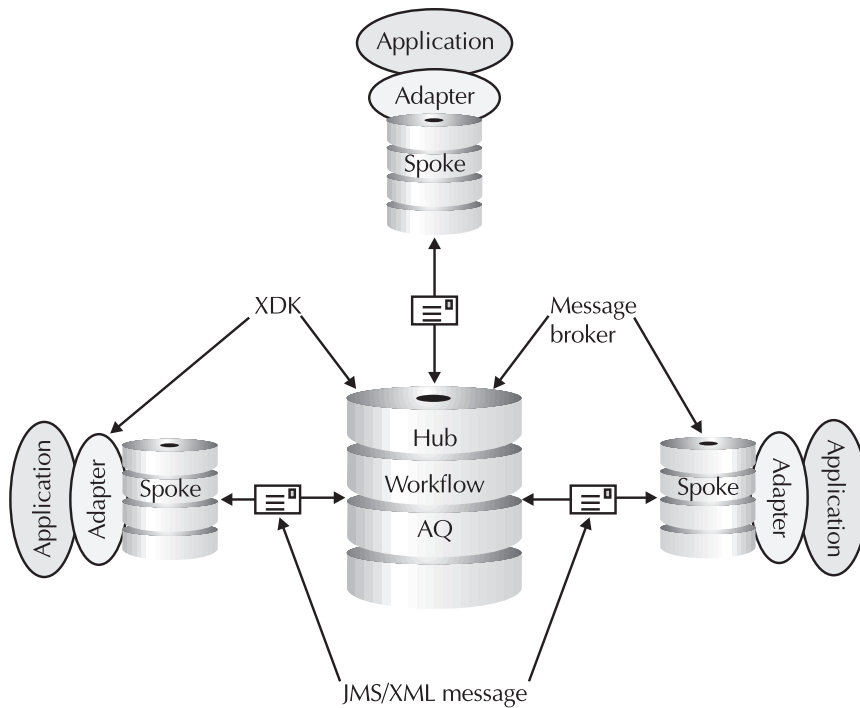
While this example showed a consumer-to-business relationship, the same technology and architecture could be used for a business-to-business site. Wholesalers and manufacturers could present their inventories for businesses to bid, or conversely, businesses could post a purchase order that would trigger messages to be sent to vendors to bid on fulfillment.

In the case of application-to-application messaging, the XML SQL Utility’s classes can be called directly to read and write data. Taking advantage of Oracle 8i’s *Advanced Queuing* (AQ) and Oracle Workflow, routing logic can be applied to prioritize and send the messages in the proper order. See the following section on the Oracle Integration Server for more details.

Oracle Integration Server

The *Oracle Integration Server (OIS)* is a suite of products designed to provide the generic XML-based messaging infrastructure that allows easy implementation of application integration across intranets or the Internet. Figure 8-10 shows the architecture of OIS. Note that this is based on a hub-and-spoke architecture that simplifies administration and management by reducing the routing. Adapters are built or provided to generate the XML message payload that is accepted by *Oracle Message Broker (OMB)*. OMB then packages the message in a Java Messaging Service envelope and routes it to the hub. This type of transport has the advantage of being transparent to firewalls, as well as providing control over service quality and resource usage.

Once the message arrives at the hub, as shown in Figure 8-11, it is queued and submitted to the rules engine within Oracle Workflow for proper routing. XSL stylesheets are utilized to transform the message into the destination’s desired format, and it is requeued and sent via OMB to one or more receiving applications.

**FIGURE 8-10.** *Oracle Integration Server architecture*

This type of messaging, enabled by the XML open standards, permits applications to be loosely coupled and to communicate asynchronously, placing the integration logic in the hub instead of hard-coded into the applications.

As can be seen from the examples, XML and databases are closely linked at many levels of application integration. The missing piece that allows this to occur seamlessly is the XML schema standard. Already, vendor-neutral groups, such as XML.org and the Open Applications Group, have begun to create repositories

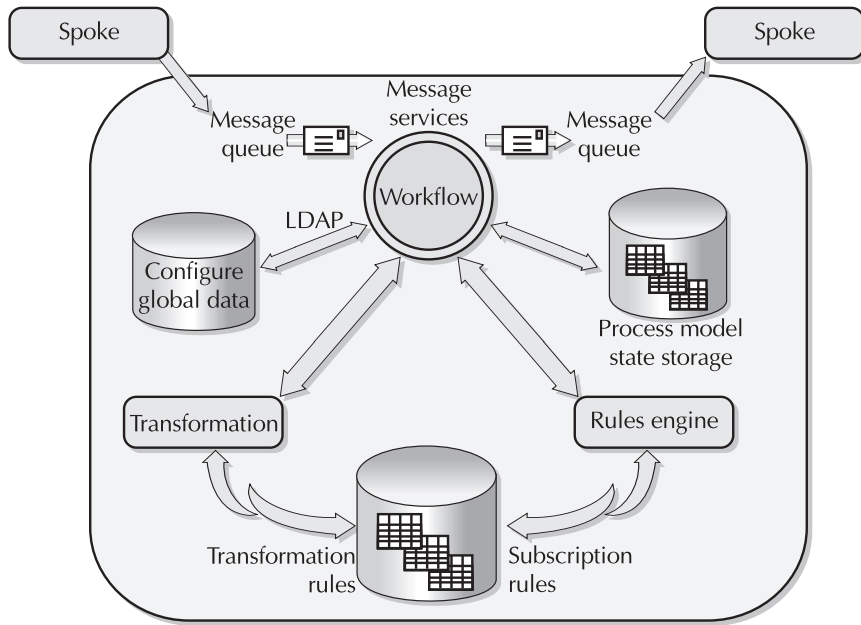


FIGURE 8-11. *Oracle Integration Server Hub architecture*

of specific industry DTDs and schemas to permit businesses in the same market segment to exchange data easily. The addition of simple and complex data types to the XML technology stack will permit precise data mappings in an open standard. This will only speed the adoption of XML as the data interchange format.